Assignment 6: CuPy-based ODE IVP Solvers

Matthew Lochridge

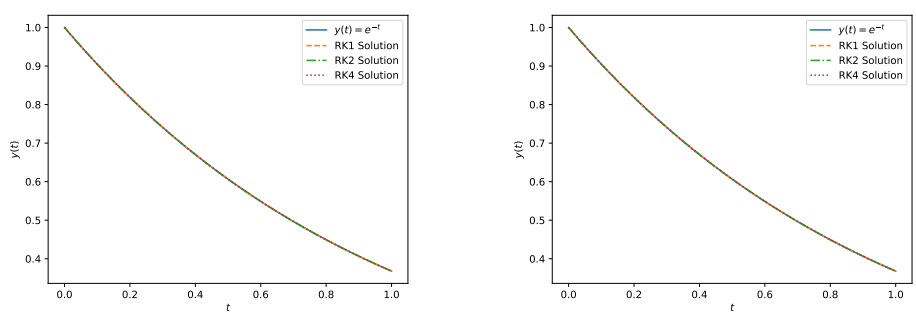
April 2026

1 Explicit Methods

1.1 Implementation

Explicit forward Euler (RK1), second-order midpoint Runge-Kutta (RK2), and fourth-order Runge-Kutta (RK4) methods were implemented in Python using the corresponding Butcher tableaus sourced from [2]. The provided backend.py code was used to make the computation automatically transferrable between serial NumPy and GPU-parallelized CuPy modules. In the CuPy case, the JIT compiler was warmed up before timing, and results were transferred to the CPU afterward. Source code and instructions are provided at [1].

These methods were applied to the nonstiff ODE IVP given by $y'(t) = -y(t)$ for $t \in [0, 1]$ with $y(0) = 1$, which has the analytical solution $y(t) = e^{-t}$. Fig. 1 shows visual validation of each solution, illustrating that the CPU and GPU implementations produce identical results and approximate the analytical solution well.



(a) (b)

Figure 1: Explicit RK solutions compared to analytical solution (**a**) on CPU, and (**b**) on GPU.

1.2 Convergence

A convergence study was run on the CPU with $\Delta t = 2^{-n}$ with $n \in \{4, 5, 6, 7, 8, 9, 10\}$. Fig. 2 shows the global error $|y(t) - e^{-t}|$ evaluated at $t = 1$ for each method over the range of Δt, along with power law fits to confirm convergence order. As expected, the convergence order of each RK method corresponds to the order of the method.

1.3 Scaling and Parallel Speedup

For the same range of Δt, the scaling of CPU and GPU implementations was studied, along with the parallel speedup. Once the IVP parameters were initialized, the RK calculations were timed using the Timer class included in the provided backend.py code. With CuPy, the GPU threads were synchronized after computation, before the timer was stopped. The scaling results are shown in Fig. 5.

As expected, runtime decreases with increasing Δt in general. In the GPU case, we see significant startup overhead when the first method is run (RK1 for $\Delta t = 2^{-4}$). Furthermore, after startup the GPU implementation still takes longer than the CPU implementation, leading to speedups less than unity (in Fig. 3d, the speedup is rounded to the nearest integer, which is zero).

1.4 Stability Regions

Now generalizing the previous ODE IVP as $y'(t) = \lambda y(t)$, and defining the complex parameter $z = \lambda \Delta t$, the stability of each RK method was studied.

Taking the definition of the stability function $R(z)$ as $y_{n+1} = R(z) y_n$, where stability requires $|R(z)| \leq 1$, [2] gives for RK methods in general $R(z) = 1 + z\mathbf{b}^T (I - zA)^{-1} \mathbf{e}$, where I is the identity matrix, $\mathbf{e}$ is the vector of ones, and $\mathbf{b}$ and A are the weights and matrix from the corresponding Butcher tableau, respectively. [3] further derives $(I - zA)^{-1} = \sum_{k=0}^{\infty} (zA)^k$ by applying the geometric series. This formally reduces to

$$R(z) = 1 + \sum_{k=0}^{\infty} \mathbf{b}^T A^k \mathbf{e} z^{k+1} \tag{1}$$

In practice, the number of terms in the sum corresponds to the order of the RK method implemented, and Eqn. 1 is the form of the stability function that was utilized in this study. The contours of the stability region boundaries are shown in Fig. 4.

As discussed in class, the theoretical stability regions are $|\lambda|\Delta t \leq 2$ for RK1 and RK2, and $|\lambda|\Delta t \leq 2.79$ for RK4. The contours in Fig. 4 (with grid lines for reference) show agreement with these theoretical values, as the largest negative value of x for which $|R(z)| \leq 1$ (corresponding to the x-intercept of the contour in the left half-plane) appears to be -2 for RK1 and RK2, and roughly -2.8 for RK4.

From the positive powers of $z = x + iy$ in Eqn. 1, we can infer that these explicit methods cannot be A-stable. As $x \to -\infty$, so does $z \to -\infty$. Returning to the definition of the stability function,

$$\lim_{z \to -\infty} y_{n+1} = \lim_{z \to -\infty} R(z) y_n = \lim_{z \to -\infty} \left(1 + \sum_{k=0}^{\infty} \mathbf{b}^T A^k \mathbf{e} z^{k+1} \right) y_n \tag{2}$$

We conclude that there always exists a large enough value of $-\mathrm{Re}(z)$ that will cause $y_{n+1} \to \pm\infty$, with the sign depending on the order of the method used.

2 GPU Speedup via Large-batch Ensemble Integration

The RK4 method was adapted to accept batches of independent trajectories and applied to the logistic equation $y'(t) = ry(t)\,(1 - y(t))$ with $r = 2$, $t \in [0, 10]$, and $\Delta t = 10^{-3}$. N equally-spaced trajectories were specified by $y(0) \in [0.01, 0.99]$, with $N \in \{10^3, 10^4, 10^5, 10^6, 10^7\}$. Source code and instructions are provided at [1].

Timing results for CPU and GPU implementations are shown in Fig. **??**. Evidently the GPU starts to outperform the CPU around $N = 60,000$, where the speedup in Fig. 5b crosses unity and the runtime and throughput curves cross one another in Figs. 5a and 5c, respectively. Kernel launching was accounted for by warming up Cupy's JIT compiler before timing, and memory transfers are not applicable since the trajectories are independent. Competition occurs between the fixed speed of individual Python loops and the benefits of increasing vectorization; smaller-scale vectorization of trajectories cannot compete with the loop speed, but large-scale vectorization can. With large enough N, we see the GPU performance begin to saturate, as many trajectories leads to high occupancy; the GPU runtime in Fig. 5a approaches linearity, the speedup in Fig. 5b approaches some upper limit, and the GPU throughput in Fig. 5c has already flattened.

3 Stiff Decay Modes

Finally, a stiff diagonal system was studied, described by

$$y_i'(t) = -\alpha_i y_i(t) \text{ for } y_i(0) = 1 \text{ and } i \in \{1, ..., d\} \tag{3}$$

with log-spaced $\alpha_i \in [1, 10^6]$. Source code and instructions are provided at [1].

3.1 Explicit Methods (RK)

As mentioned in Sec. 1.4 for explicit RK methods, the step size must be small enough to resolve the eigenvalue $|\lambda_i| = \alpha_i$. Specifically, for RK1 and RK2, $\Delta t_i \leq 2/\alpha_i$, and for RK4 $\Delta t_i \leq 2.79/\alpha_i$. In the current implementation, with fixed Δt for the system of d trajectories, this requires $\Delta t \sim 2/\alpha_{\max} = 2 \times 10^{-6}$ just to ensure that the stiff decay modes are stable. Fig. 6a demonstrates the instability of RK methods with $\Delta t = 0.5$ and increasing stiffness.

3.2 Implicit Methods (TR and TRBDF2)

In contrast, implicit methods such as the trapezoidal rule (TR) and TR followed by second-order backward differentiation (BDF2) are better equipped to handle stiff systems. Even so, Fig. 6b illustrates that TR cannot strongly damp the stiff decay modes as TRBDF2 does when Δt is large.

3.3 GPU Benchmark

To take advantage of the GPU, we set $d = 10^6$ and run TRBDF2 with a range of $\Delta t \in [2 \times 10^{-6}, 2 \times 10^{-1}]$, comparing performance to the CPU implementation. The results are summarized in Fig. 7.

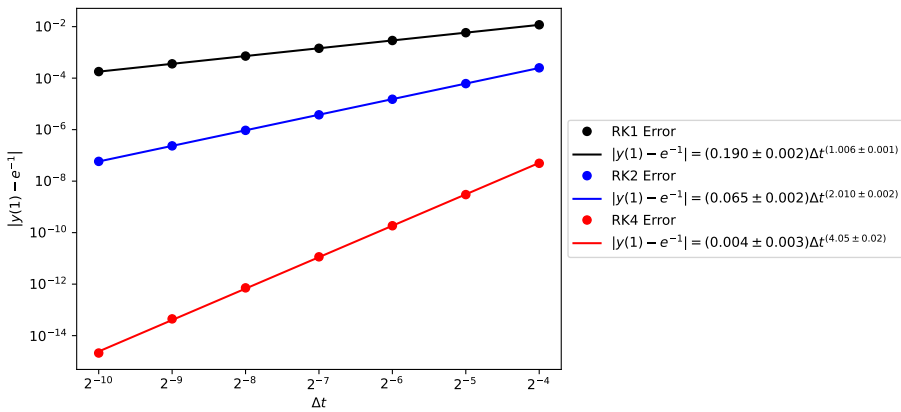
We see that for smaller Δt, the GPU overhead dominates the runtime, even past the point where the GPU outperforms the CPU. Eventually, around $\Delta t \sim 10^{-4}$, the GPU runtime begins to scale logarithmically similar to the CPU runtime. Thus for fine-grained integrations, CuPy is the more efficient choice, showing considerable speedup compared to NumPy. In the summary table, we note that TRBDF2 is preferable to TR because the stability is not severely impacted by the step size, as seen in Fig. 6b.

References

[1] HPC-Spring-2026/hw6 on GitHub.

[2] Runge-Kutta Methods.

[3] 4.3. Stability Function of a Runge-Kutta Method — Runge-Kutta methods.

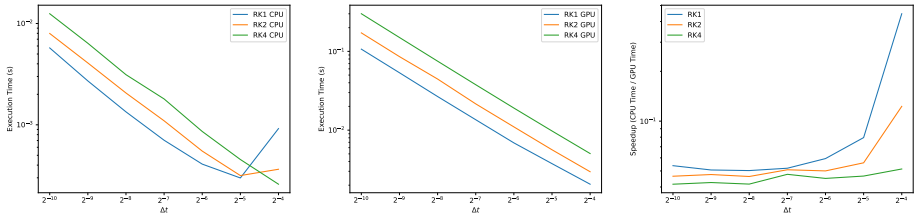


(a)

Method	Convergence Order	Notes
RK1	1.006	Uncertainty = ±0.001
RK2	2.010	Uncertainty = ±0.002
RK4	4.05	Uncertainty = ±0.02

(b)

Figure 2: (**a**) Global error of each RK method versus Δt with power law fits. (**b**) Convergence table summarizing results for each RK method.



(a) (b) (c)

Method	CPU Time (s)	GPU Time (s)	Speedup (CPU/GPU)	Notes
RK1	0.005742	0.106703	0	
RK2	0.007974	0.171637	0	
RK4	0.012492	0.300468	0	

(d)

Figure 3: (**a**, **b**) Execution time of each RK method on CPU and GPU, respectively. (**c**) GPU speedup. (**d**) Scaling table summarizing results of (**c**) for $\Delta t = 2^{-10}$.

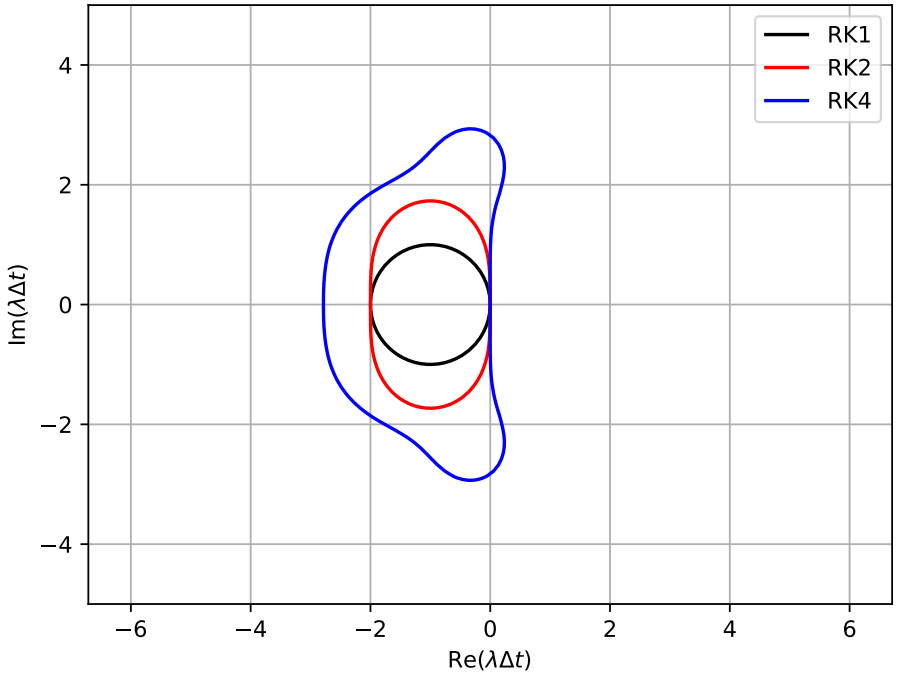
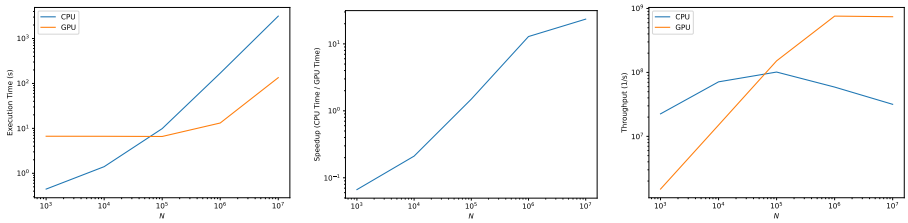
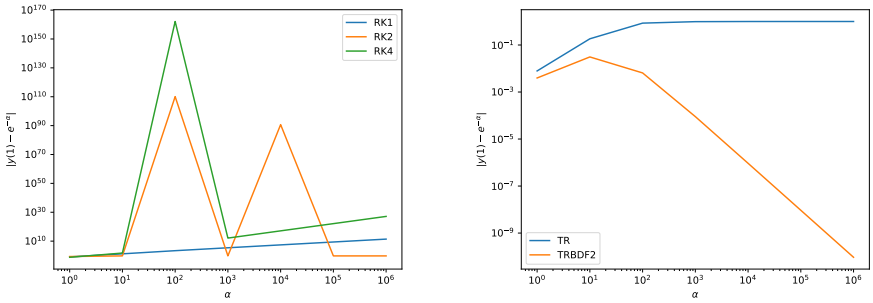


Figure 4: Stability region boundaries for each RK method.



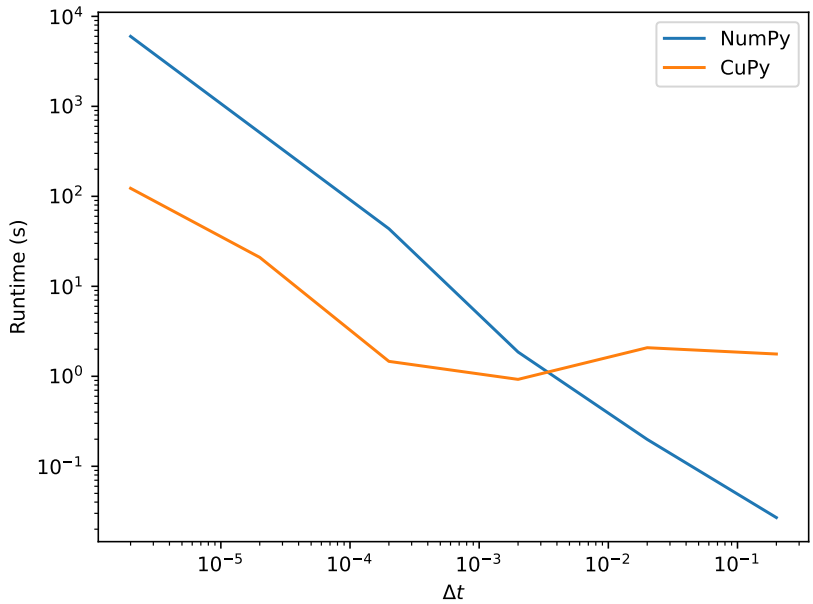
(a) (b) (c)

Figure 5: (**a**) Execution time of batched RK4 method on the logistic equation using CPU and GPU. (**b**) GPU speedup. (**b**) Throughput ($N\times$ number of steps / runtime) of CPU and GPU implementations.



(a) (b)

Figure 6: Global error vs. α using $\Delta t = 0.5$ with (**a**) explicit methods and (**b**) implicit methods. $d = 7$ independent trajectories were used in this example, with timescales differing by an order of magnitude.



Method	CPU Time (s)	GPU Time (s)	Speedup	Max. Error	Notes
TR with large Δt				0.9999840001279992	Not stable for large α
TR with small Δt					Stable for large α
TRBDF2 with large Δt	0.026918	1.769254	0.015214	9.32513863824999e-11	Stable for large α
TRBDF2 with small Δt	5995.340959	122.849001	48.802521		Stable for large α

Figure 7: (**a**) Runtime vs. Δt for stiff system using TRBDF2 on CPU and GPU. (**b**) Benchmark table summarizing results of TR and TRBDF2 tests.